

Food App Backend API - Endpoints Reference

This document provides a concise reference for the API endpoints implemented in the Food App Backend.

The API is designed for CRUD (Create, Read, Update, Delete) operations on various food-related entities.

Base URL

All API endpoints are accessible from: `http://localhost:5000`

4. API Endpoints Reference

You can interact with these endpoints using any HTTP client tool, such as Postman, Insomnia, or `curl`.

For `POST` and `PUT` requests, ensure you set the `Content-Type` header to `application/json` in your HTTP client.

4.1. User Management (`/api/users`)

This section details the API endpoints for managing `User` resources.

- **Model Fields:** `user_id`, `username` (unique), `email` (unique), `password_hash` (internal, hashed), `dietary_preferences`, `date_joined`.

Retrieve All Users

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/users`
- **Description:** Fetches a list of all registered users in the system.
- **Response (200 OK):**

```
[
  {
    "user_id": 1,
    "username": "john.doe",
```

```

        "email": "john.doe@example.com",
        "dietary_preferences": "Omnivore",
        "date_joined": "2024-01-15T10:30:00.000Z"
    },
    {
        "user_id": 2,
        "username": "jane.smith",
        "email": "jane.smith@example.com",
        "dietary_preferences": "Vegetarian",
        "date_joined": "2024-02-20T14:45:00.000Z"
    }
]

```

Create a New User

- **Method:** `POST`
- **URL:** `http://localhost:5000/api/users`
- **Description:** Registers a new user account. `username`, `email`, and `password` are mandatory fields.
- **Request Body (JSON):**

```

{
    "username": "newuser_example",
    "email": "newuser@example.com",
    "password": "strong_secure_password_123!",
    "dietary_preferences": "Pescatarian"
}

```

- **Required Fields:** `username` (string, unique), `email` (string, unique, valid email format), `password` (string, min 6 characters).
- **Optional Fields:** `dietary_preferences` (string, allowed values: "Vegan", "Vegetarian", "Pescatarian", "Omnivore", "Gluten-Free", "Keto"). If omitted, it will be `null`.

- **Response (201 Created):**

```

{
    "user_id": 3,
    "username": "newuser_example",
    "email": "newuser@example.com",
    "dietary_preferences": "Pescatarian",
    "date_joined": "2024-06-12T15:00:00.000Z"
}

```

- **Error Responses (Examples):**

- 400 Bad Request: {"message": "Missing username, email, or password"} (if required fields are missing)
- 409 Conflict: {"message": "Username already exists"} (if username is not unique)
- 409 Conflict: {"message": "Email already exists"} (if email is not unique)
- 400 Bad Request: {"message": "{ 'password': ['Password must be at least 6 characters long'] }"} (if password is too short or other validation fails)
- 400 Bad Request: {"message": "{ 'dietary_preferences': ['Invalid dietary preference.'] }"} (if dietary preference is not one of the allowed values)

Retrieve User by ID

- **Method:** GET
- **URL:** `http://localhost:5000/api/users/<user_id>` (e.g., `http://localhost:5000/api/users/1`)
- **Description:** Fetches detailed information for a single user identified by their unique `user_id`.
- **Response (200 OK):**

```
{
  "user_id": 1,
  "username": "john.doe",
  "email": "john.doe@example.com",
  "dietary_preferences": "Omnivore",
  "date_joined": "2024-01-15T10:30:00.000Z"
}
```

- **Error Response (404 Not Found):**

```
{
  "message": "User with ID <user_id> not found."
}
```

Update User by ID

- **Method:** PUT
- **URL:** `http://localhost:5000/api/users/<user_id>` (e.g., `http://localhost:5000/api/users/1`)

- **Description:** Updates existing user information. Accepts partial JSON data, allowing specific fields to be modified without sending all fields.
- **Request Body (JSON - Example for partial update):**

```
{
  "dietary_preferences": "Keto",
  "email": "john.updated@example.com"
}
```

- **Updatable Fields:** `username` (string), `email` (string), `dietary_preferences` (string).
- **(Note:** *This endpoint does not support changing the password directly for security reasons; password updates typically require a separate, more secure endpoint involving current password verification or reset tokens.*)
- **Response (200 OK):**

```
{
  "message": "User updated successfully",
  "user": {
    "user_id": 1,
    "username": "john.doe",
    "email": "john.updated@example.com",
    "dietary_preferences": "Keto",
    "date_joined": "2024-01-15T10:30:00.000Z"
  }
}
```

- **Error Responses (Examples):**
 - 404 Not Found: `{"message": "User with ID <user_id> not found."}`
 - 409 Conflict: `{"message": "Username already exists"}` (if updating `username` to an existing one)
 - 409 Conflict: `{"message": "Email already exists"}` (if updating `email` to an existing one)
 - 400 Bad Request: `{"message": "{ 'email': ['Not a valid email address.'] }"}` (if email format is invalid)

Delete User by ID

- **Method:** `DELETE`
- **URL:** `http://localhost:5000/api/users/<user_id>` (e.g., `http://localhost:5000/api/users/1`)
- **Description:** Deletes a user record from the database.

- **Response (204 No Content):** No response body; indicates successful deletion.
-

4.2. Food Category Management (`/api/categories`)

This section details the API endpoints for managing `FoodCategory` resources.

- **Model Fields:** `category_id`, `name` (unique), `description`.

Retrieve All Food Categories

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/categories`
- **Description:** Fetches a list of all available food categories.
- **Response (200 OK):**

```
[
  {
    "category_id": 1,
    "name": "Breakfast",
    "description": "Meals typically eaten in the morning."
  },
  {
    "category_id": 2,
    "name": "Dinner",
    "description": "Main meals for the evening."
  }
]
```

Create a New Food Category

- **Method:** `POST`
- **URL:** `http://localhost:5000/api/categories`
- **Description:** Creates a new food category. `name` is mandatory.
- **Request Body (JSON):**

```
{
  "name": "Snacks",
  "description": "Light bites between meals."
}
```

- **Required Fields:** `name` (string, unique).

- **Optional Fields:** `description` (string).

- **Response (201 Created):**

```
{
  "category_id": 3,
  "name": "Snacks",
  "description": "Light bites between meals."
}
```

- **Error Responses (Examples):**

- `400 Bad Request`: `{"message": "Missing category name"}`
- `409 Conflict`: `{"message": "Category name already exists"}`

Retrieve Food Category by ID

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/categories/<category_id>` (e.g., `http://localhost:5000/api/categories/1`)
- **Description:** Fetches detailed information for a single food category by its unique `category_id`.
- **Response (200 OK):**

```
{
  "category_id": 1,
  "name": "Breakfast",
  "description": "Meals typically eaten in the morning."
}
```

- **Error Response (404 Not Found):**

```
{
  "message": "Food category with ID <category_id> not found."
}
```

Update Food Category by ID

- **Method:** `PUT`
- **URL:** `http://localhost:5000/api/categories/<category_id>` (e.g., `http://localhost:5000/api/categories/1`)
- **Description:** Updates an existing food category's information. Accepts partial JSON data.
- **Request Body (JSON):**

```
{
  "description": "Morning meals to start the day, often consumed be
}
```

(You can update *name* or *description*.)

- **Response (200 OK):**

```
{
  "message": "Category updated successfully",
  "category": {
    "category_id": 1,
    "name": "Breakfast",
    "description": "Morning meals to start the day, often consume
  }
}
```

- **Error Responses (Examples):**

- 404 Not Found: {"message": "Food category with ID <category_id> not found."}
- 409 Conflict: {"message": "Category name already exists"} (if updating *name* to an existing one)

Delete Food Category by ID

- **Method:** DELETE
- **URL:** `http://localhost:5000/api/categories/<category_id>` (e.g., `http://localhost:5000/api/categories/1`)
- **Description:** Deletes a food category record.
- **Response (204 No Content):** (No response body)

4.3. Food Item Management (/api/foods)

This section details the API endpoints for managing *Food* resources.

- **Model Fields:** *food_id*, *name*, *description*, *nutritional_info* (JSONB), *category_id* (foreign key to *FoodCategory*), *category* (nested object for category details).

Retrieve All Food Items

- **Method:** GET

- **URL:** `http://localhost:5000/api/foods`
- **Description:** Fetches a list of all available food items, including their associated category details.
- **Response (200 OK):**

```
[
  {
    "food_id": 101,
    "name": "Oatmeal with Berries",
    "description": "Healthy breakfast with rolled oats and mixed
    "nutritional_info": {"calories": 300, "protein_g": 10, "carbs
    "category_id": 1,
    "category": {
      "category_id": 1,
      "name": "Breakfast"
    }
  }
]
```

Create a New Food Item

- **Method:** `POST`
- **URL:** `http://localhost:5000/api/foods`
- **Description:** Creates a new food item. `name` and `category_id` are mandatory.
- **Request Body (JSON):**

```
{
  "name": "Grilled Salmon with Asparagus",
  "description": "Delicious and healthy grilled salmon fillet serve
  "nutritional_info": {
    "calories": 450,
    "protein_g": 40,
    "fat_g": 25,
    "carbs_g": 10
  },
  "category_id": 2
}
```

- **Required Fields:** `name` (string, unique), `category_id` (integer, must exist in `FoodCategory`).
- **Optional Fields:** `description` (string), `nutritional_info` (JSON object, flexible key-value pairs like `calories` , `protein_g` , etc.).

- **Dependency:** The `category_id` must correspond to an existing `FoodCategory` record.

- **Response (201 Created):**

```
{
  "food_id": 102,
  "name": "Grilled Salmon with Asparagus",
  "description": "Delicious and healthy grilled salmon fillet served with asparagus",
  "nutritional_info": {"calories": 450, "protein_g": 40, "fat_g": 20, "carbs_g": 10},
  "category_id": 2,
  "category": {
    "category_id": 2,
    "name": "Dinner"
  }
}
```

- **Error Responses (Examples):**

- 400 Bad Request: {"message": "Missing food name or category ID"}
- 404 Not Found: {"message": "Food category not found with the provided ID"} (if `category_id` doesn't exist)
- 409 Conflict: {"message": "Food with this name already exists"} (if `name` is not unique)

Retrieve Food Item by ID

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/foods/<food_id>` (e.g., `http://localhost:5000/api/foods/101`)
- **Description:** Fetches detailed information for a single food item by its unique `food_id`. Includes associated category.
- **Response (200 OK):**

```
{
  "food_id": 101,
  "name": "Oatmeal with Berries",
  "description": "Healthy breakfast with rolled oats and mixed berries",
  "nutritional_info": {"calories": 300, "protein_g": 10, "carbs_g": 50, "fat_g": 5},
  "category_id": 1,
  "category": {
    "category_id": 1,
    "name": "Breakfast"
  }
}
```

```
}  
}
```

- **Error Response (404 Not Found):**

```
{  
  "message": "Food with ID <food_id> not found."  
}
```

Update Food Item by ID

- **Method:** PUT
- **URL:** `http://localhost:5000/api/foods/<food_id>` (e.g.,
`http://localhost:5000/api/foods/101`)
- **Description:** Updates an existing food item's information. Accepts partial JSON data.
- **Request Body (JSON):**

```
{  
  "description": "Hearty breakfast with organic rolled oats, fresh  
  "nutritional_info": {  
    "calories": 320,  
    "protein_g": 12,  
    "carbs_g": 55,  
    "fiber_g": 8  
  },  
  "category_id": 1  
}
```

- **Updatable Fields:** `name` (string), `description` (string), `nutritional_info` (JSON object), `category_id` (integer).
- (**Note:** If updating `category_id`, ensure it references an existing `FoodCategory`.)
- **Response (200 OK):**

```
{  
  "message": "Food updated successfully",  
  "food": {  
    "food_id": 101,  
    "name": "Oatmeal with Berries",  
    "description": "Hearty breakfast with organic rolled oats, fr  
    "nutritional_info": {"calories": 320, "protein_g": 12, "carbs  
    "category_id": 1,  
    "category": {
```

```

        "category_id": 1,
        "name": "Breakfast"
    }
}

```

- **Error Responses (Examples):**

- 404 Not Found: {"message": "Food with ID <food_id> not found."}
- 404 Not Found: {"message": "Food category not found with the provided ID"} (if updating `category_id` to a non-existent one)
- 409 Conflict: {"message": "Food with this name already exists"} (if updating `name` to an existing one)

Delete Food Item by ID

- **Method:** DELETE
- **URL:** `http://localhost:5000/api/foods/<food_id>` (e.g., `http://localhost:5000/api/foods/101`)
- **Description:** Deletes a food item record from the database.
- **Response (204 No Content):** (No response body)

4.4. Ingredient Management (`/api/ingredients`)

This section details the API endpoints for managing `Ingredient` resources.

- **Model Fields:** `ingredient_id`, `name` (unique), `description`.

Retrieve All Ingredients

- **Method:** GET
- **URL:** `http://localhost:5000/api/ingredients`
- **Description:** Fetches a list of all available ingredients.
- **Response (200 OK):**

```

[
  {
    "ingredient_id": 1,
    "name": "Eggs",
    "description": "Common protein source, versatile in cooking."
  }
]

```

Create a New Ingredient

- **Method:** `POST`
- **URL:** `http://localhost:5000/api/ingredients`
- **Description:** Creates a new ingredient. `name` is mandatory.
- **Request Body (JSON):**

```
{
  "name": "Spinach",
  "description": "Leafy green vegetable, rich in iron."
}
```

- **Required Fields:** `name` (string, unique).
- **Optional Fields:** `description` (string).
- **Response (201 Created):**

```
{
  "ingredient_id": 2,
  "name": "Spinach",
  "description": "Leafy green vegetable, rich in iron."
}
```

- **Error Responses (Examples):**
 - `400 Bad Request: {"message": "Missing ingredient name"}`
 - `409 Conflict: {"message": "Ingredient with this name already exists"}`

Retrieve Ingredient by ID

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/ingredients/<ingredient_id>` (e.g., `http://localhost:5000/api/ingredients/1`)
- **Description:** Fetches detailed information for a single ingredient by its unique `ingredient_id`.
- **Response (200 OK):**

```
{
  "ingredient_id": 1,
  "name": "Eggs",
  "description": "Common protein source, versatile in cooking."
}
```

- **Error Response (404 Not Found):**

```
{
  "message": "Ingredient with ID <ingredient_id> not found."
}
```

Update Ingredient by ID

- **Method:** PUT
- **URL:** `http://localhost:5000/api/ingredients/<ingredient_id>` (e.g., `http://localhost:5000/api/ingredients/1`)
- **Description:** Updates an existing ingredient's information. Accepts partial JSON data.
- **Request Body (JSON):**

```
{
  "description": "Cage-free organic eggs, excellent for baking and
```

(You can update *name* or *description*.)

- **Response (200 OK):**

```
{
  "message": "Ingredient updated successfully",
  "ingredient": {
    "ingredient_id": 1,
    "name": "Eggs",
    "description": "Cage-free organic eggs, excellent for baking
```

- **Error Responses (Examples):**

- 404 Not Found: `{"message": "Ingredient with ID <ingredient_id> not found."}`
- 409 Conflict: `{"message": "Ingredient with this name already exists"}` (if updating *name* to an existing one)

Delete Ingredient by ID

- **Method:** DELETE
- **URL:** `http://localhost:5000/api/ingredients/<ingredient_id>` (e.g., `http://localhost:5000/api/ingredients/1`)

- **Description:** Deletes an ingredient record.
- **Response (204 No Content):** (No response body)

4.5. Food Image Management (/api/food_images)

This section details the API endpoints for managing `FoodImage` resources.

- **Model Fields:** `image_id`, `food_id` (foreign key to `Food`), `image_url`, `description`.

Retrieve All Food Images

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/food_images`
- **Description:** Fetches a list of all stored food image records.
- **Response (200 OK):**

```
[
  {
    "image_id": 1,
    "food_id": 101,
    "image_url": "[https://example.com/images/oatmeal_bowl.jpg] (1)",
    "description": "Overhead shot of oatmeal with berries."
  }
]
```

Create a New Food Image

- **Method:** `POST`
- **URL:** `http://localhost:5000/api/food_images`
- **Description:** Creates a new food image entry. `food_id` and `image_url` are mandatory.
- **Request Body (JSON):**

```
{
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_ingredients.jpg] (1)",
  "description": "Ingredients for oatmeal preparation."
}
```

- **Required Fields:** `food_id` (integer, must exist in `Food`), `image_url` (string, valid URL format).
- **Optional Fields:** `description` (string).

- **Dependency:** The `food_id` must correspond to an existing `Food` item.

- **Response (201 Created):**

```
{
  "image_id": 2,
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_ingredients.jpg",
  "description": "Ingredients for oatmeal preparation."
}
```

- **Error Responses (Examples):**

- 400 Bad Request: {"message": "Missing food ID or image URL"}
- 404 Not Found: {"message": "Food item not found with the provided ID"} (if `food_id` doesn't exist)
- 400 Bad Request: {"message": "{ 'image_url': ['Not a valid URL. '] }"} (if image URL is not a valid URL)

Retrieve Food Image by ID

- **Method:** `GET`
- **URL:** `http://localhost:5000/api/food_images/<image_id>` (e.g., `http://localhost:5000/api/food_images/1`)
- **Description:** Fetches detailed information for a single food image by its unique `image_id`.
- **Response (200 OK):**

```
{
  "image_id": 1,
  "food_id": 101,
  "image_url": "[https://example.com/images/oatmeal_bowl.jpg] (https://example.com/images/oatmeal_bowl.jpg)",
  "description": "Overhead shot of oatmeal with berries."
}
```

- **Error Response (404 Not Found):**

```
{
  "message": "Food Image with ID <image_id> not found."
}
```

Update Food Image by ID

- **Method:** `PUT`

- **URL:** `http://localhost:5000/api/food_images/<image_id>` (e.g., `http://localhost:5000/api/food_images/1`)
- **Description:** Updates an existing food image's information. Accepts partial JSON data.
- **Request Body (JSON):**

```
{
  "image_url": "[https://example.com/images/oatmeal_serving.jpg] (ht
  "description": "Image of oatmeal served on a table."
}
```

- **Updatable Fields:** `food_id` (integer), `image_url` (string), `description` (string).
- (**Note:** If updating `food_id`, ensure it references an existing `Food` item.)
- **Response (200 OK):**

```
{
  "message": "Food image updated successfully",
  "food_image": {
    "image_id": 1,
    "food_id": 101,
    "image_url": "[https://example.com/images/oatmeal_serving.jpg
    "description": "Image of oatmeal served on a table."
  }
}
```

- **Error Responses (Examples):**
 - 404 Not Found: `{"message": "Food Image with ID <image_id> not found."}`
 - 404 Not Found: `{"message": "Food item not found with the provided ID"}` (if updating `food_id` to a non-existent one)
 - 400 Bad Request: `{"message": "'{'image_url': ['Not a valid URL. ']}'"}` (if image URL is not a valid URL)

Delete Food Image by ID

- **Method:** `DELETE`
- **URL:** `http://localhost:5000/api/food_images/<image_id>` (e.g., `http://localhost:5000/api/food_images/1`)
- **Description:** Deletes a food image record.
- **Response (204 No Content):** (No response body)

5. Stopping the Application

To cleanly stop and remove the running Docker containers and their associated Docker network, navigate to the project root directory (`FOOD_APP`) in your terminal and execute:

```
docker compose down
```